Name:Gawade Payal [23107030] [Batch:A]

Name:Burpalle Krushnai [23107010] [Batch:A]

PARKINSONS DISEASE DATA EXPLORATION AND MODELING

```
#  Import Libraries
# Core libraries
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')  # For environments without GUI
import matplotlib.pyplot as plt
plt.switch_backend('Agg')
%matplotlib inline
import seaborn as sns

# Machine learning libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
roc_curve, auc
from sklearn.ensemble import RandomForestClassifier

# Save the trained model
import joblib

sns.set(style='whitegrid')
print('Imports loaded successfully')
```

```
Imports loaded successfully
```

```
#  Load Dataset

data_path = 'parkinsons_disease_dataset.csv'  # Path to your dataset
df = pd.read_csv(data_path, encoding='ascii', delimiter=',')

print('Dataset shape:', df.shape)
print(df.head())
print(df.tail())
```

```
Dataset shape: (23000, 15)
   Age     Sex  Tremor_Intensity  Voice_Pitch  Voice_Jitter
Voice_Shimmer  \
0   78    Male              5.19       287.62          3.82
2.98
1   68  Female              6.22       281.92          3.38
3.79
2   54  Female              4.20       231.59          2.89
4.03
```

```
3   82    Male              8.77       236.50          3.16
2.89
4   47    Male              5.23       100.48          4.60
3.43

   Bradykinesia_Score  Postural_Instability  Rigidity_Level
UPDRS_Score  \
0                8.05                  9.11            6.67
79.13
1                1.04                  7.64            1.43
97.50
2                8.14                  2.05            3.80
73.40
3                5.35                  6.52            4.87
87.81
4                6.80                  8.13            3.59
94.33

   Handwriting_Change  Walking_Difficulty Family_History  MRI_Score  \
0                0.97                7.39             No      11.41
1                1.67                1.71            Yes      12.58
2                6.30                9.79            Yes       1.55
3                3.41                2.33             No      92.83
4                6.41                0.52            Yes      36.37

   Parkinsons_Status
0                  1
1                  0
2                  0
3                  1
4                  0
       Age     Sex  Tremor_Intensity  Voice_Pitch  Voice_Jitter  \
22995   81  Female              4.10       102.12          0.72
22996   65    Male              0.25       290.78          0.12
22997   81  Female              4.19       254.47          4.97
22998   80    Male              6.60       239.11          2.31
22999   41  Female              8.91       105.96          0.44

       Voice_Shimmer  Bradykinesia_Score  Postural_Instability  \
22995           3.93                7.18                  9.45
22996           2.29                0.32                  2.43
22997           3.70                4.79                  0.79
22998           1.60                4.90                  8.03
22999           0.59                0.43                  2.30

       Rigidity_Level  UPDRS_Score  Handwriting_Change
Walking_Difficulty  \
22995            9.81        46.06                5.65
9.90
22996            6.39         8.81                3.94
```

```
5.09
22997            6.68        23.80                9.71
4.38
22998            6.94        35.20                9.72
9.04
22999            9.77        72.33                9.25
5.59

      Family_History  MRI_Score  Parkinsons_Status
22995            Yes      36.29                  0
22996             No      16.44                  0
22997             No      53.74                  0
22998             No      82.09                  1
22999             No      70.50                  1
```

```
# Data Cleaning and Preprocessing

# Check for missing values
print('\nMissing values in each column:')
print(df.isnull().sum())

# Convert categorical columns to category type
df['Sex'] = df['Sex'].astype('category')
df['Family_History'] = df['Family_History'].astype('category')

# Display statistical summary
print('\nStatistical Summary:')
print(df.describe())

# Create dummy variables for categorical data
df_processed = pd.get_dummies(df, columns=['Sex', 'Family_History'],
drop_first=True)
print('\nProcessed dataset shape:', df_processed.shape)
```

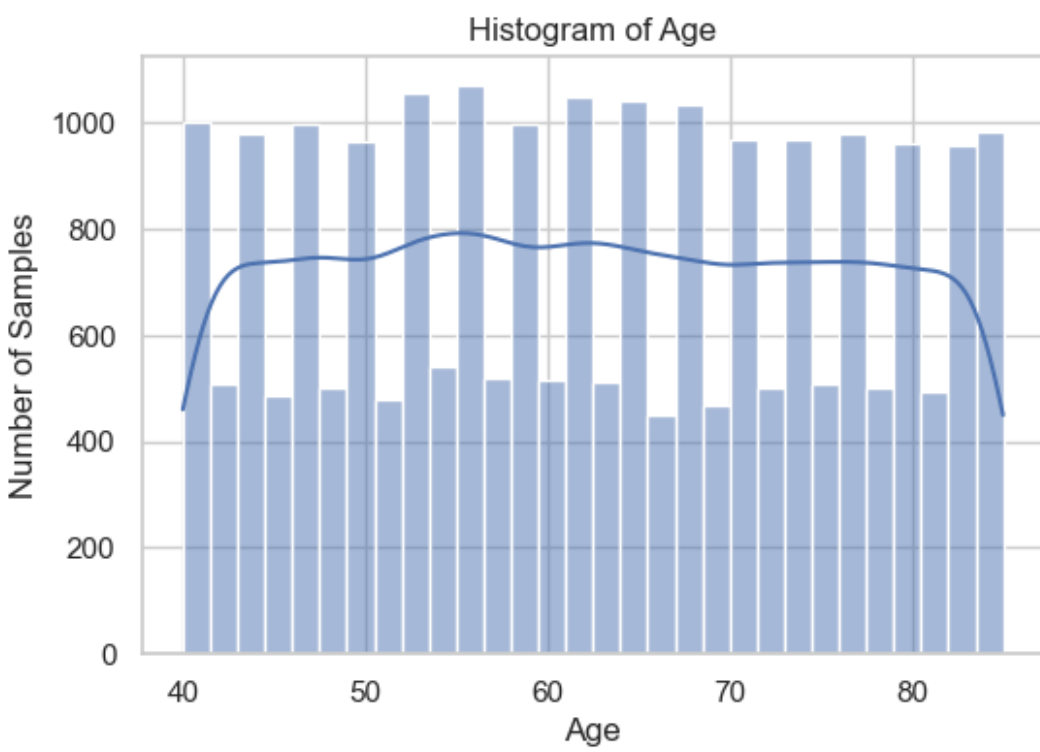
```
Missing values in each column:
Age                      0
Sex                      0
Tremor_Intensity         0
Voice_Pitch              0
Voice_Jitter             0
Voice_Shimmer            0
Bradykinesia_Score       0
Postural_Instability     0
Rigidity_Level           0
UPDRS_Score              0
Handwriting_Change       0
Walking_Difficulty       0
Family_History           0
MRI_Score                0
```

```
Parkinsons_Status       0
dtype: int64

Statistical Summary:
                Age  Tremor_Intensity   Voice_Pitch  Voice_Jitter  \
count  23000.000000      23000.000000  23000.000000   23000.00000
mean      62.352783          4.965473    200.423182       2.54379
std       13.192461          2.885786     57.487770       1.41737
min       40.000000          0.000000    100.000000       0.10000
25%       51.000000          2.450000    151.207500       1.30000
50%       62.000000          4.950000    200.600000       2.56000
75%       74.000000          7.470000    250.465000       3.78000
max       85.000000         10.000000    299.990000       5.00000

       Voice_Shimmer  Bradykinesia_Score  Postural_Instability  \
count   23000.000000        23000.000000          23000.000000
mean        2.551544            4.999794              5.032178
std         1.416150            2.883474              2.891638
min         0.100000            0.000000              0.000000
25%         1.320000            2.500000              2.520000
50%         2.560000            5.000000              5.035000
75%         3.780000            7.490000              7.540000
max         5.000000           10.000000             10.000000

       Rigidity_Level   UPDRS_Score  Handwriting_Change
Walking_Difficulty  \
count    23000.000000  23000.000000        23000.000000
23000.000000
mean         5.009445     49.758583            5.039512
4.986073
std          2.877759     28.936998            2.891534
2.887473
min          0.000000      0.000000            0.000000
0.000000
25%          2.547500     24.280000            2.530000
2.490000
50%          5.010000     49.770000            5.040000
4.970000
75%          7.480000     74.832500            7.530000
7.470000
max         10.000000    100.000000           10.000000
10.000000

          MRI_Score  Parkinsons_Status
count  23000.000000       23000.000000
mean      50.063531           0.399696
std       28.885844           0.489846
min        0.010000           0.000000
25%       25.027500           0.000000
50%       50.300000           0.000000
```

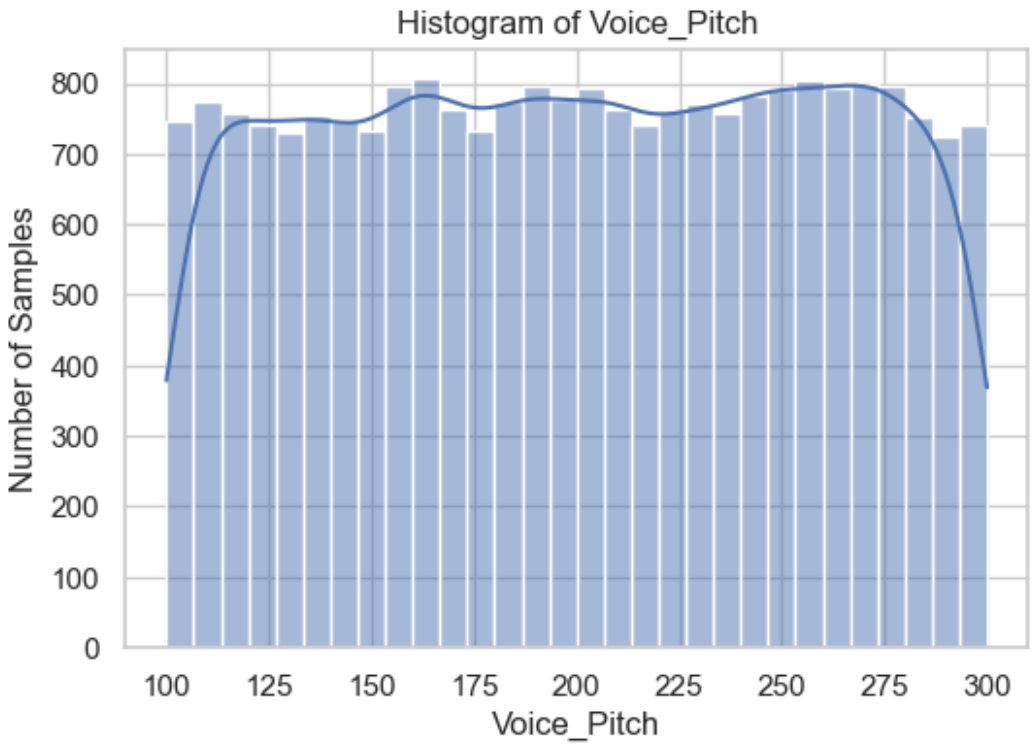
```
75%        75.242500            1.000000
max        99.990000            1.000000
```

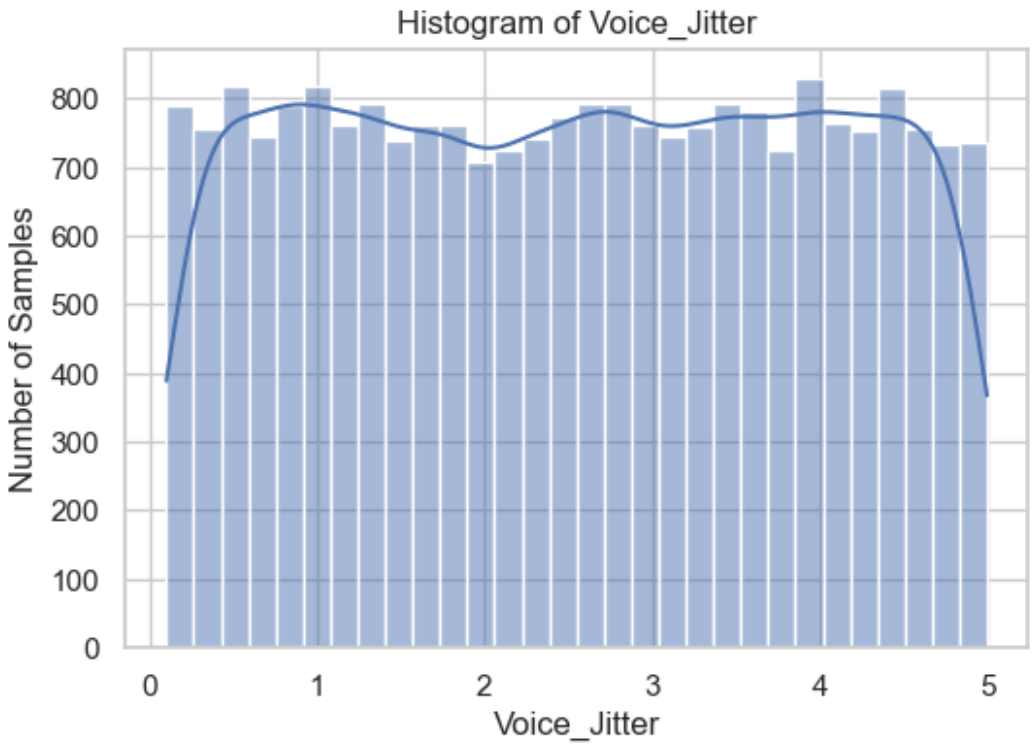
```
Processed dataset shape: (23000, 15)
```

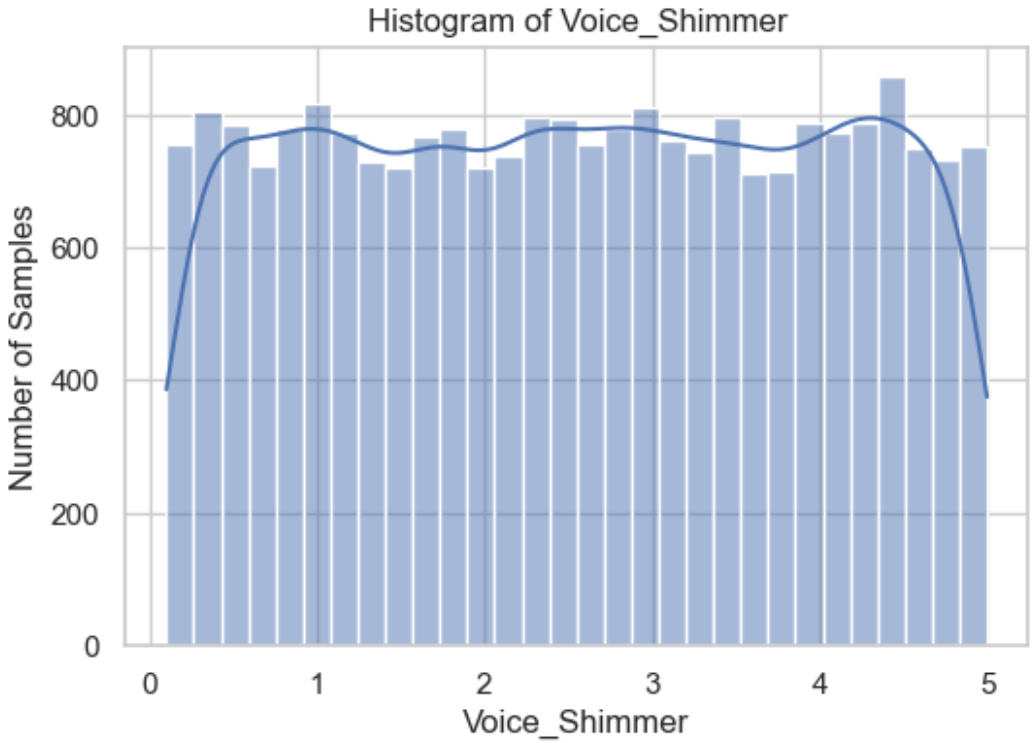
```python
#  Exploratory Data Analysis (EDA)
# Histograms for numeric columns
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
for col in numeric_cols:
    plt.figure(figsize=(6, 4))
    sns.histplot(df[col], kde=True, bins=30,stat='count')
    plt.title(f'Histogram of {col}')
    plt.xlabel(col)
    plt.ylabel('Number of Samples')
    plt.show()

# Count plot for target variable
plt.figure(figsize=(6,4))
sns.countplot(x='Parkinsons_Status', data=df)
plt.title('Count Plot for Parkinsons_Status')
plt.xlabel('Parkinsons_Status')
plt.ylabel('Count')
plt.show()
```

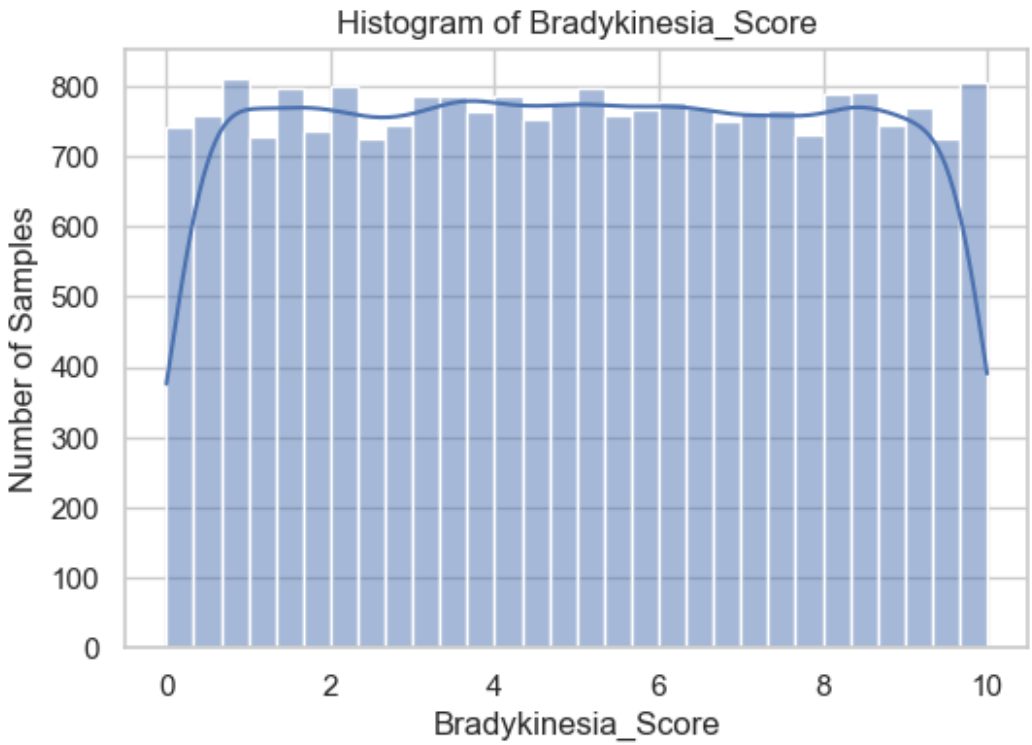


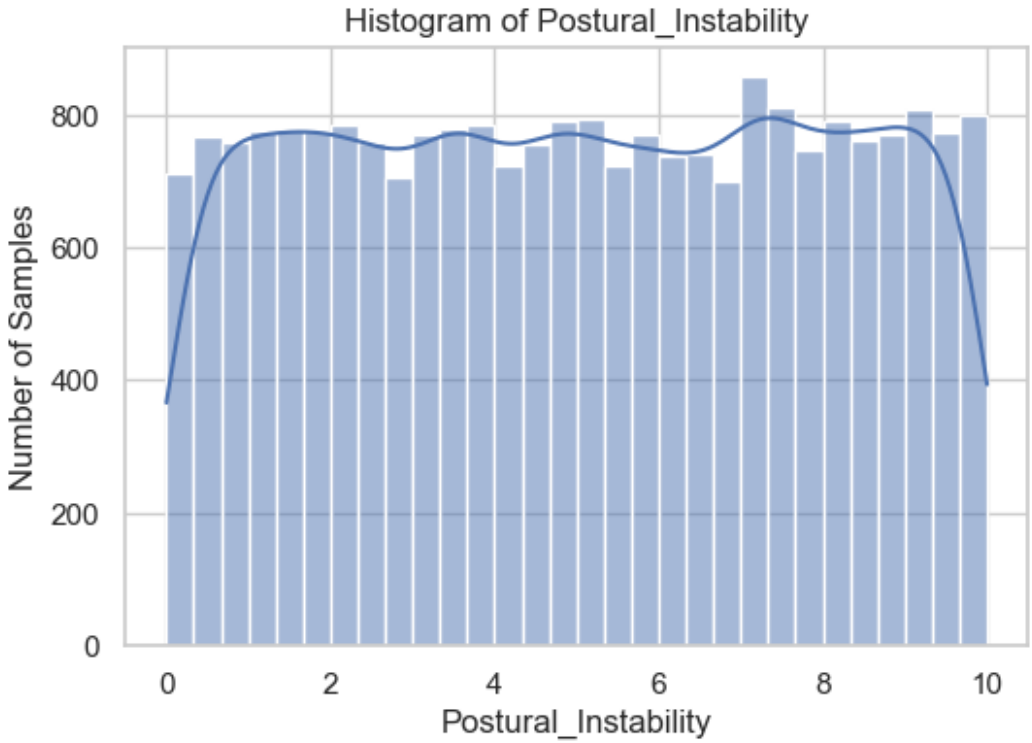
Histogram of Tremor_Intensity
Number of Samples
0
100
200
300
400
500
600
700
800
0
2
4
6
8
10
Tremor_Intensity

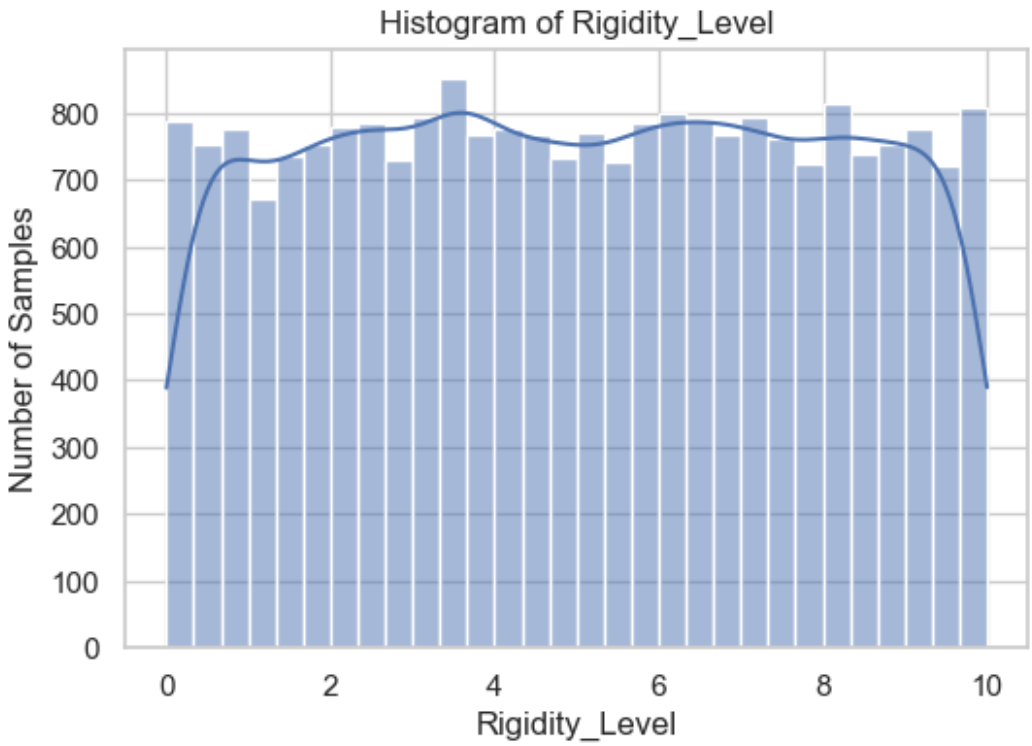

Histogram of Voice_Pitch
Number of Samples
0
100
200
300
400
500
600
700
800
100
125
150
175
200
225
250
275
300
Voice_Pitch

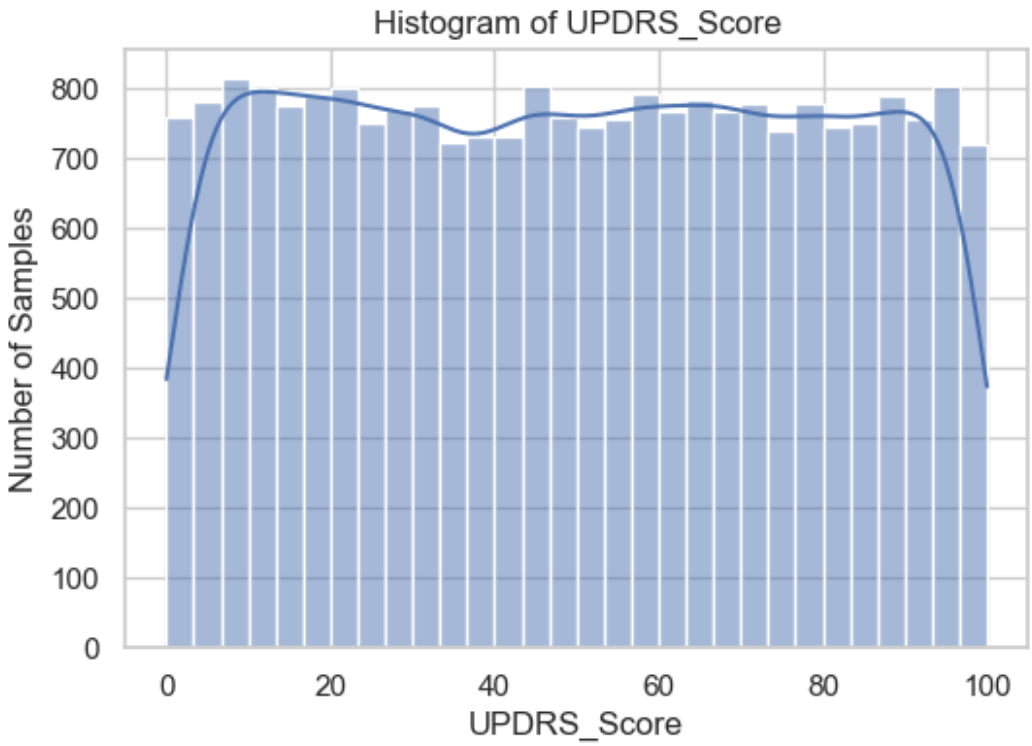

Histogram of Voice_Jitter
Number of Samples
0
100
200
300
400
500
600
700
800
0
1
2
3
4
5
Voice_Jitter

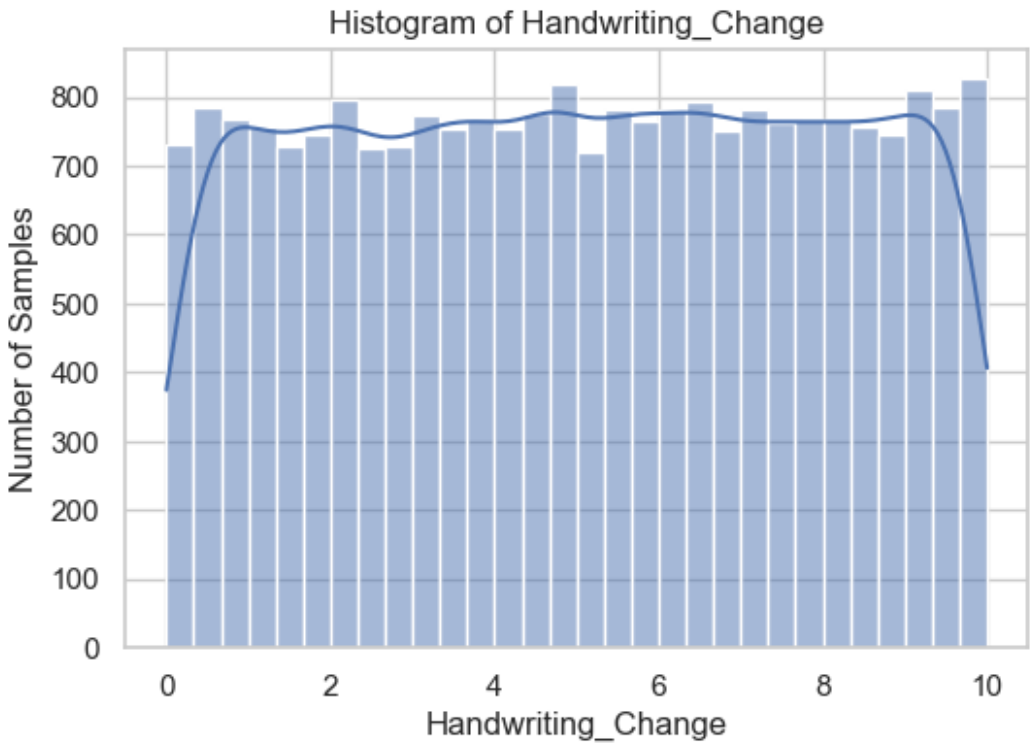

Histogram of Voice_Shimmer
Number of Samples
0
200
400
600
800
0
1
2
3
4
5
Voice_Shimmer

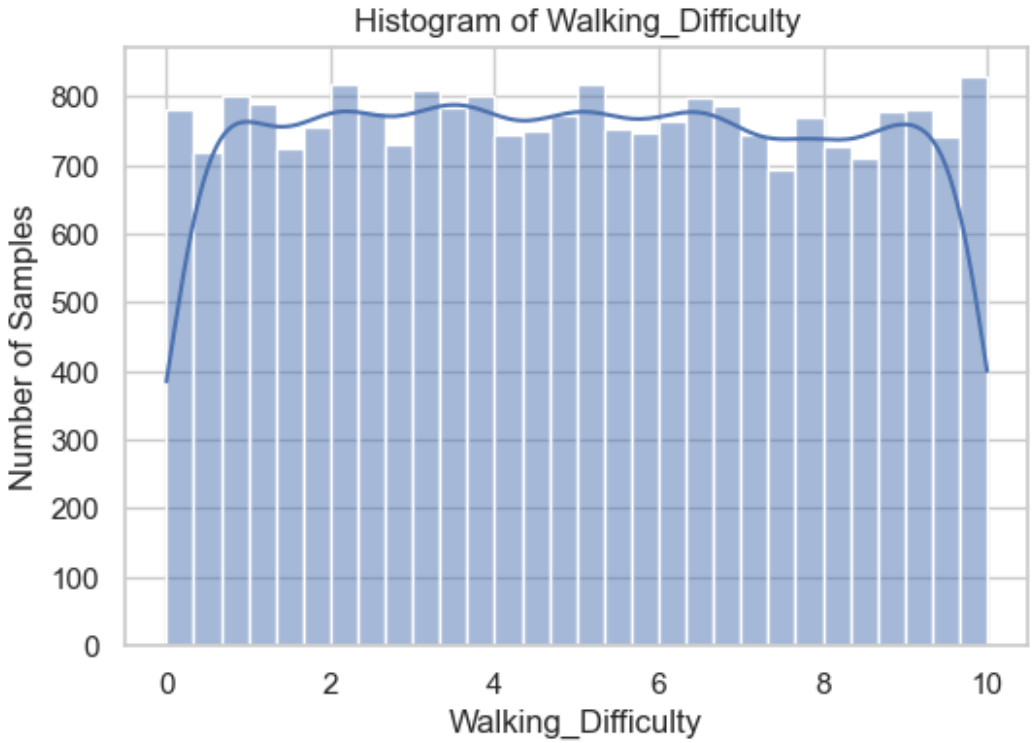

Histogram of Bradykinesia_Score
Number of Samples
0
100
200
300
400
500
600
700
800
0
2
4
6
8
10
Bradykinesia_Score

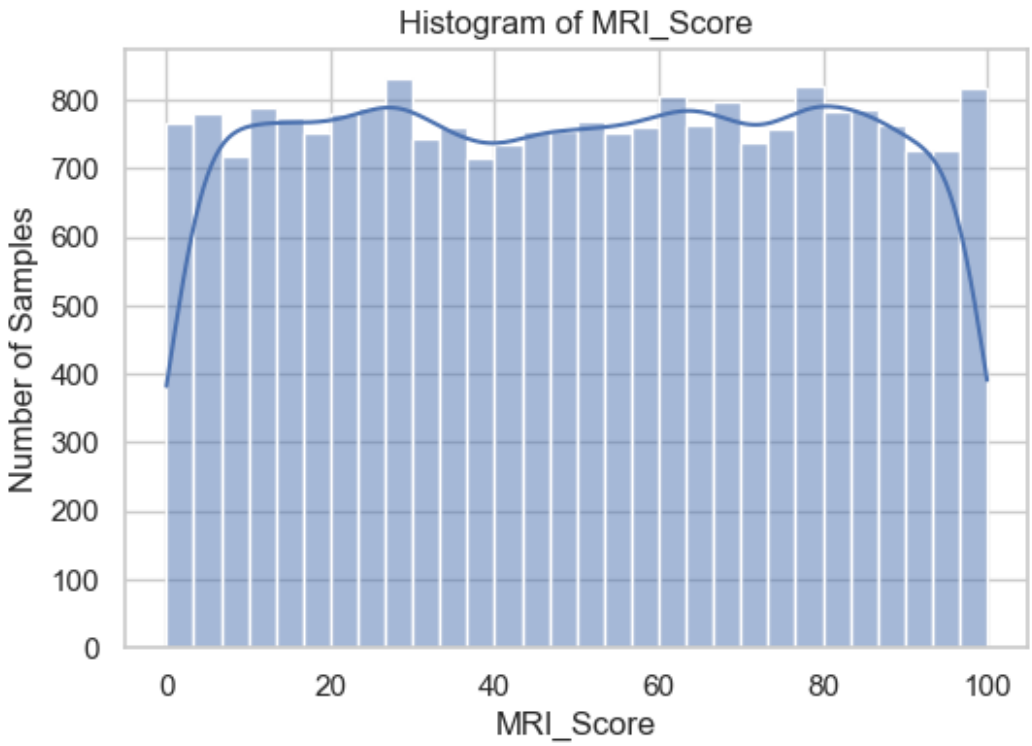

Histogram of Postural_Instability
Number of Samples
0
200
400
600
800
0
2
4
6
8
10
Postural_Instability

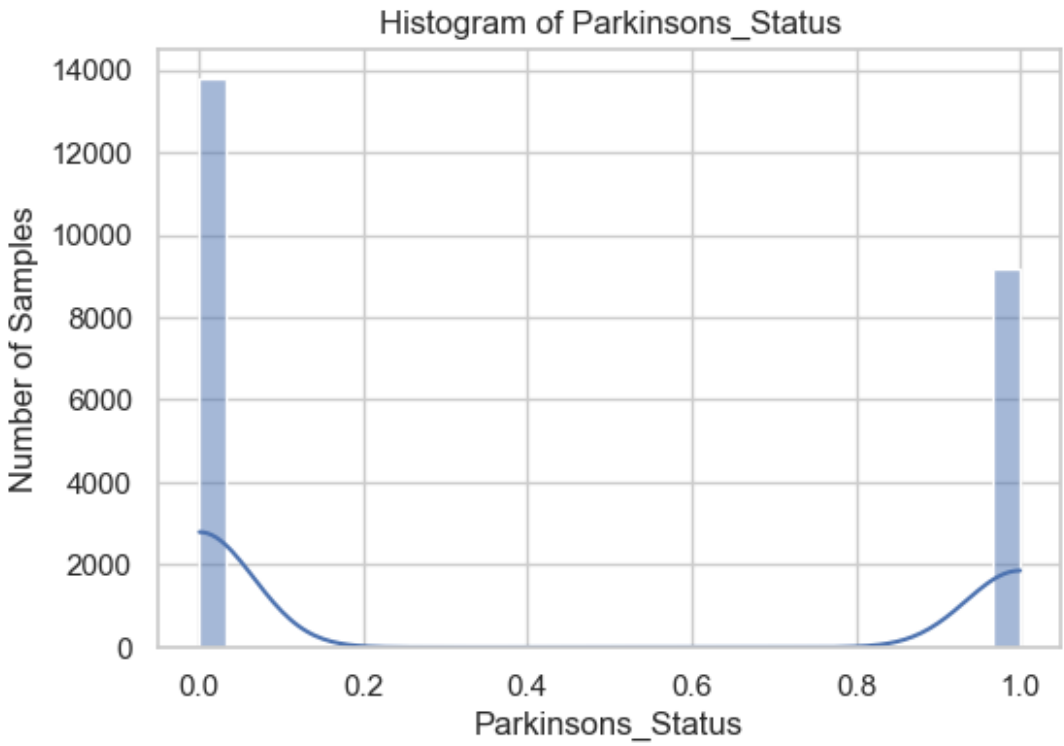

Histogram of Rigidity_Level
Number of Samples
0
100
200
300
400
500
600
700
800
0
2
4
6
8
10
Rigidity_Level

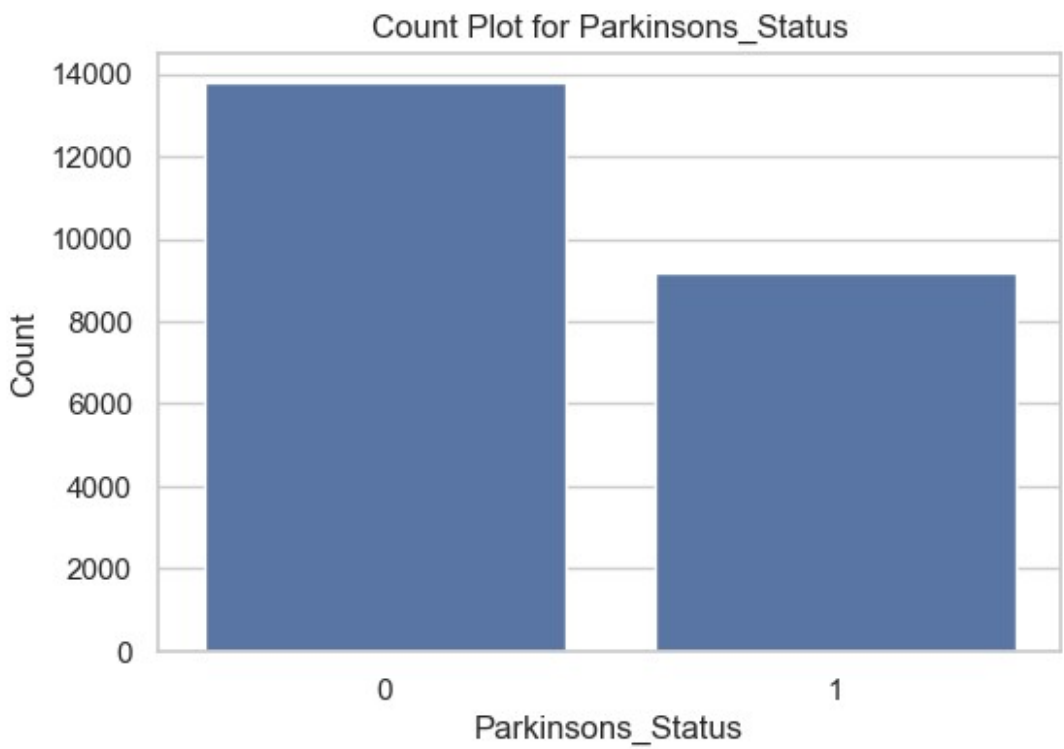

Histogram of UPDRS_Score
Number of Samples
0
100
200
300
400
500
600
700
800
0
20
40
60
80
100
UPDRS_Score


Histogram of Handwriting_Change
Number of Samples
0
100
200
300
400
500
600
700
800
0
2
4
6
8
10
Handwriting_Change

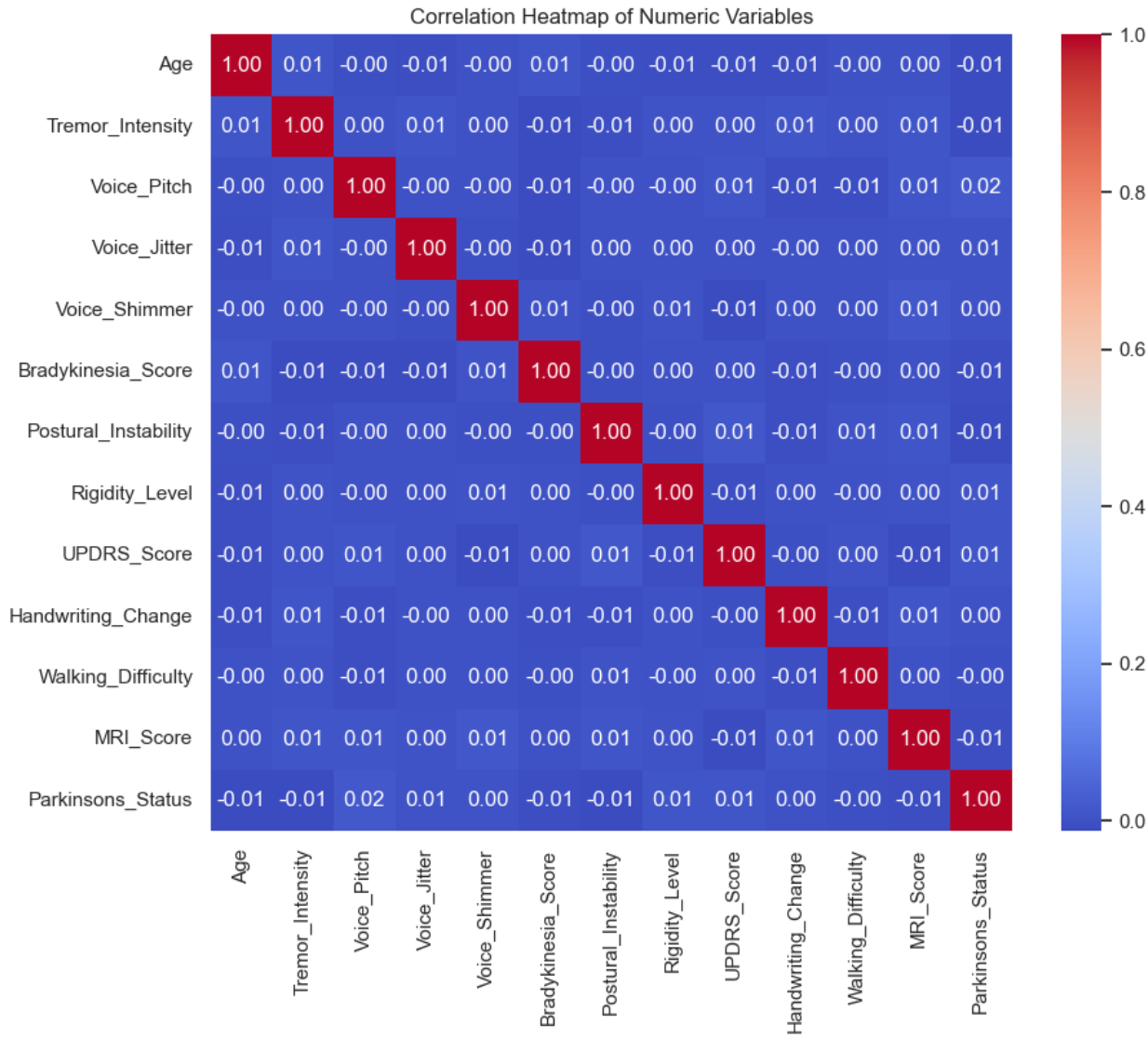

Histogram of Walking_Difficulty
Number of Samples
0
100
200
300
400
500
600
700
800
0
2
4
6
8
10
Walking_Difficulty


Histogram of MRI_Score
Number of Samples
0
100
200
300
400
500
600
700
800
0
20
40
60
80
100
MRI_Score


Histogram of Parkinsons_Status
Number of Samples
0
2000
4000
6000
8000
10000
12000
14000
0.0
0.2
0.4
0.6
0.8
1.0
Parkinsons_Status




```
#Correlation Heatmap
numeric_df = df.select_dtypes(include=[np.number])
if numeric_df.shape[1] >= 4:
    plt.figure(figsize=(10, 8))
    corr = numeric_df.corr()
    sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
    plt.title('Correlation Heatmap of Numeric Variables')
    plt.show()
```




```python
# ------------------------------------------------------------
# Data Segmentation (Updated for 500-sample blocks)
# ------------------------------------------------------------

# Separate features and target
X = df_processed.drop('Parkinsons_Status', axis=1)
y = df_processed['Parkinsons_Status']

# Break dataset into chunks of 500 samples
chunk_size = 500
num_chunks = len(X) // chunk_size

print(f"Total samples: {len(X)}")
print(f"Total chunks of {chunk_size}: {num_chunks}")

# Loop through each chunk and train/test separately
```

```
for i in range(num_chunks):
    print(f"\n=== Processing Chunk {i+1}/{num_chunks} ===")

    # Select 500-sample segment
    start_idx = i * chunk_size
    end_idx = start_idx + chunk_size
    X_chunk = X.iloc[start_idx:end_idx]
    y_chunk = y.iloc[start_idx:end_idx]

    # Split into train (70%) and test (30%) for each chunk
    X_train, X_test, y_train, y_test = train_test_split(
        X_chunk, y_chunk, test_size=0.3, random_state=42,
stratify=y_chunk
    )

    # Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Train Random Forest
    rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
    rf_model.fit(X_train_scaled, y_train)

    # Evaluate
    y_pred = rf_model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    print(f"Chunk {i+1} Accuracy: {acc*100:.2f}%")

# Summary of segmentation
print("\nData Segmentation Complete:")
print(f"Test set size: {X_test.shape[0]} samples")
```

```
Total samples: 23000
Total chunks of 500: 46

=== Processing Chunk 1/46 ===
Chunk 1 Accuracy: 55.33%

=== Processing Chunk 2/46 ===
Chunk 2 Accuracy: 54.67%

=== Processing Chunk 3/46 ===
Chunk 3 Accuracy: 53.33%

=== Processing Chunk 4/46 ===
Chunk 4 Accuracy: 56.67%

=== Processing Chunk 5/46 ===
```

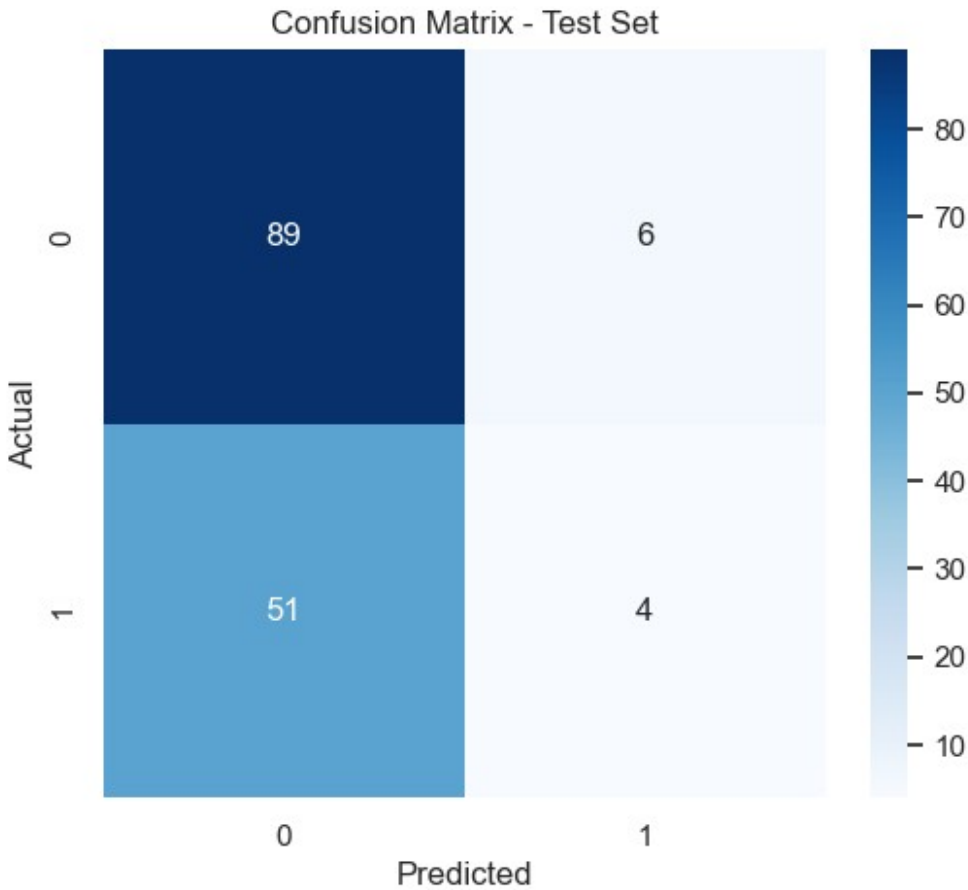
```
Chunk 5 Accuracy: 57.33%

=== Processing Chunk 6/46 ===
Chunk 6 Accuracy: 64.67%

=== Processing Chunk 7/46 ===
Chunk 7 Accuracy: 56.67%

=== Processing Chunk 8/46 ===
Chunk 8 Accuracy: 50.67%

=== Processing Chunk 9/46 ===
Chunk 9 Accuracy: 50.67%

=== Processing Chunk 10/46 ===
Chunk 10 Accuracy: 57.33%

=== Processing Chunk 11/46 ===
Chunk 11 Accuracy: 52.67%

=== Processing Chunk 12/46 ===
Chunk 12 Accuracy: 58.00%

=== Processing Chunk 13/46 ===
Chunk 13 Accuracy: 59.33%

=== Processing Chunk 14/46 ===
Chunk 14 Accuracy: 56.00%

=== Processing Chunk 15/46 ===
Chunk 15 Accuracy: 56.00%

=== Processing Chunk 16/46 ===
Chunk 16 Accuracy: 54.67%

=== Processing Chunk 17/46 ===
Chunk 17 Accuracy: 55.33%

=== Processing Chunk 18/46 ===
Chunk 18 Accuracy: 48.00%

=== Processing Chunk 19/46 ===
Chunk 19 Accuracy: 61.33%

=== Processing Chunk 20/46 ===
Chunk 20 Accuracy: 50.00%

=== Processing Chunk 21/46 ===
Chunk 21 Accuracy: 55.33%

=== Processing Chunk 22/46 ===
```

```
Chunk 22 Accuracy: 52.00%

=== Processing Chunk 23/46 ===
Chunk 23 Accuracy: 55.33%

=== Processing Chunk 24/46 ===
Chunk 24 Accuracy: 59.33%

=== Processing Chunk 25/46 ===
Chunk 25 Accuracy: 60.00%

=== Processing Chunk 26/46 ===
Chunk 26 Accuracy: 54.00%

=== Processing Chunk 27/46 ===
Chunk 27 Accuracy: 58.67%

=== Processing Chunk 28/46 ===
Chunk 28 Accuracy: 58.00%

=== Processing Chunk 29/46 ===
Chunk 29 Accuracy: 54.67%

=== Processing Chunk 30/46 ===
Chunk 30 Accuracy: 57.33%

=== Processing Chunk 31/46 ===
Chunk 31 Accuracy: 58.00%

=== Processing Chunk 32/46 ===
Chunk 32 Accuracy: 62.00%

=== Processing Chunk 33/46 ===
Chunk 33 Accuracy: 53.33%

=== Processing Chunk 34/46 ===
Chunk 34 Accuracy: 54.00%

=== Processing Chunk 35/46 ===
Chunk 35 Accuracy: 54.67%

=== Processing Chunk 36/46 ===
Chunk 36 Accuracy: 60.00%

=== Processing Chunk 37/46 ===
Chunk 37 Accuracy: 58.67%

=== Processing Chunk 38/46 ===
Chunk 38 Accuracy: 60.00%

=== Processing Chunk 39/46 ===
```

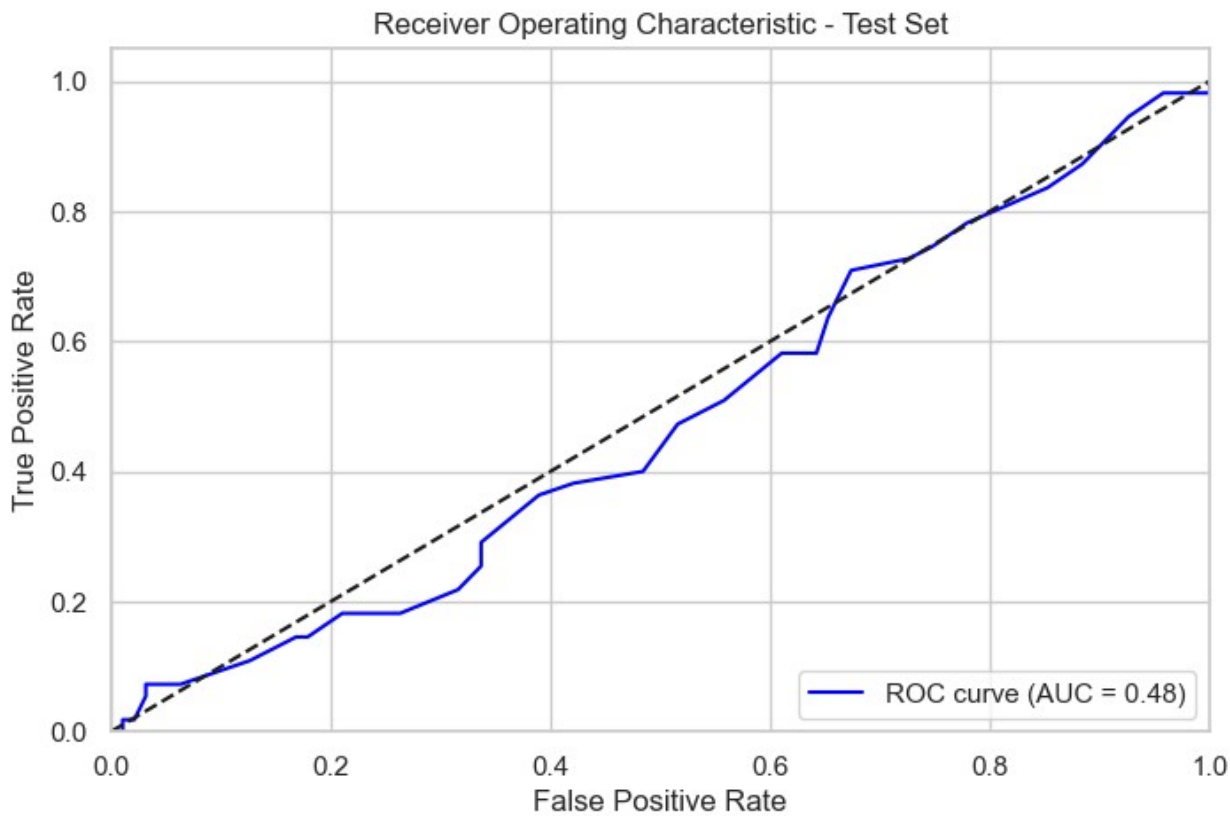
```
Chunk 39 Accuracy: 54.67%

=== Processing Chunk 40/46 ===
Chunk 40 Accuracy: 54.67%

=== Processing Chunk 41/46 ===
Chunk 41 Accuracy: 56.67%

=== Processing Chunk 42/46 ===
Chunk 42 Accuracy: 58.00%

=== Processing Chunk 43/46 ===
Chunk 43 Accuracy: 58.67%

=== Processing Chunk 44/46 ===
Chunk 44 Accuracy: 65.33%

=== Processing Chunk 45/46 ===
Chunk 45 Accuracy: 56.67%

=== Processing Chunk 46/46 ===
Chunk 46 Accuracy: 62.00%

Data Segmentation Complete:
Test set size: 150 samples
```

```python
#Final Evaluation on Test Set
y_test_pred = rf_model.predict(X_test_scaled)
test_accuracy = accuracy_score(y_test, y_test_pred)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Test Set')
plt.show()
```

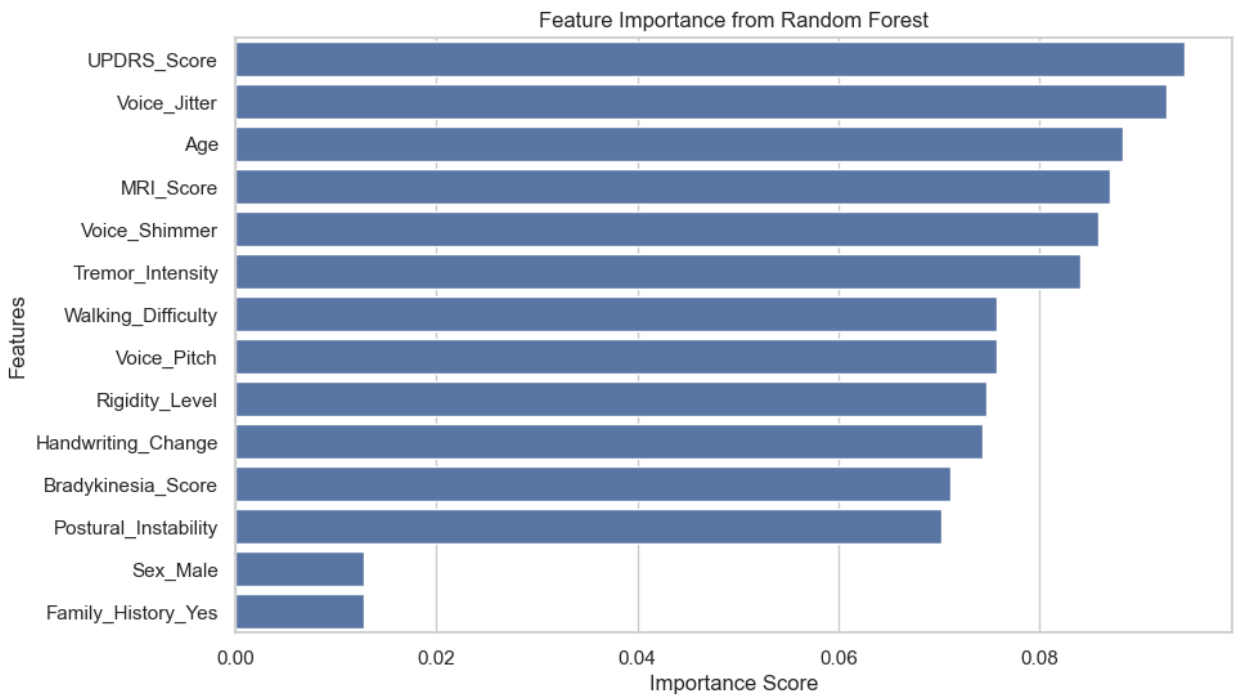
```
Test Accuracy: 62.00%
```




```
# ROC Curve

y_pred_prob = rf_model.predict_proba(X_test_scaled)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8,5))
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.2f})',
color='blue')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic - Test Set')
plt.legend(loc='lower right')
plt.show()
```




```
# Feature Importance

feature_importances = pd.Series(rf_model.feature_importances_,
index=X.columns)
sorted_importances = feature_importances.sort_values(ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(x=sorted_importances, y=sorted_importances.index)
plt.title('Feature Importance from Random Forest')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()

print("\nTop 5 Most Important Features:")
print(sorted_importances.head())
```




```
Top 5 Most Important Features:
UPDRS_Score      0.094481
Voice_Jitter     0.092638
Age              0.088331
MRI_Score        0.086983
Voice_Shimmer    0.085843
dtype: float64
```

```
# Save the Model

joblib.dump(rf_model, "parkinsons_rf_model.pkl")
joblib.dump(X.columns.tolist(), "feature_names.pkl")
print("\nModel and feature names saved successfully!")
```

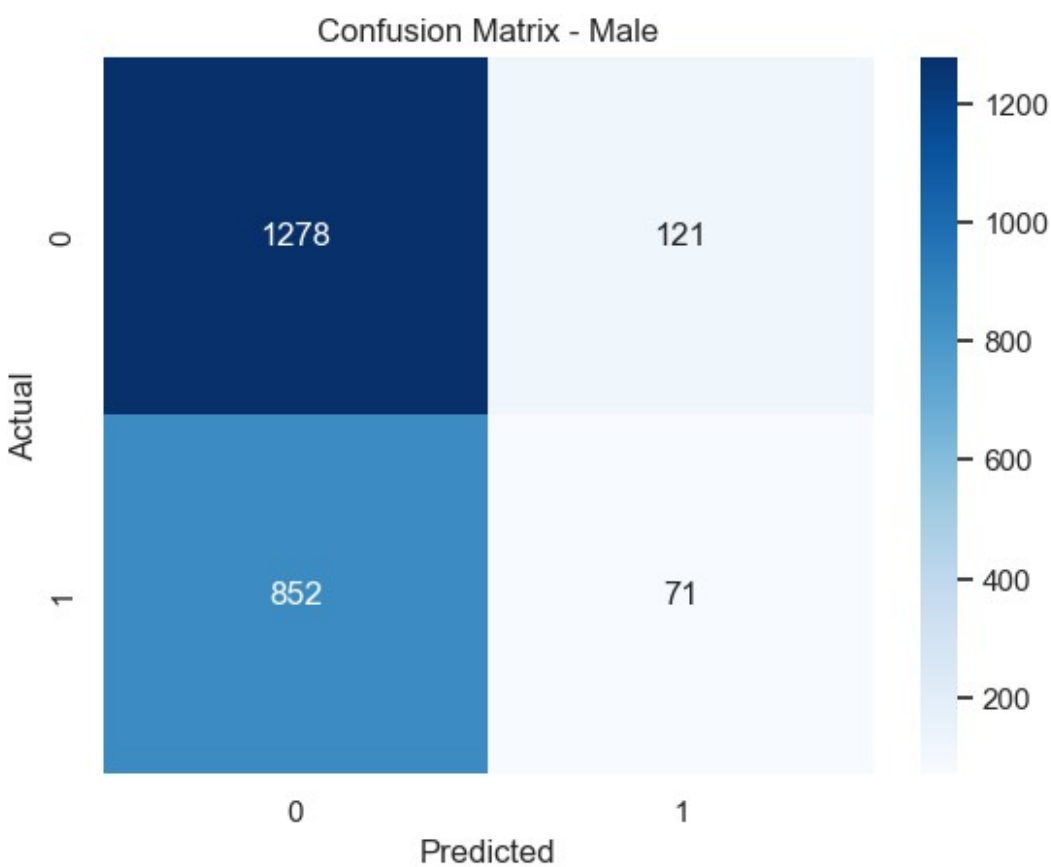
```
Model and feature names saved successfully!
```

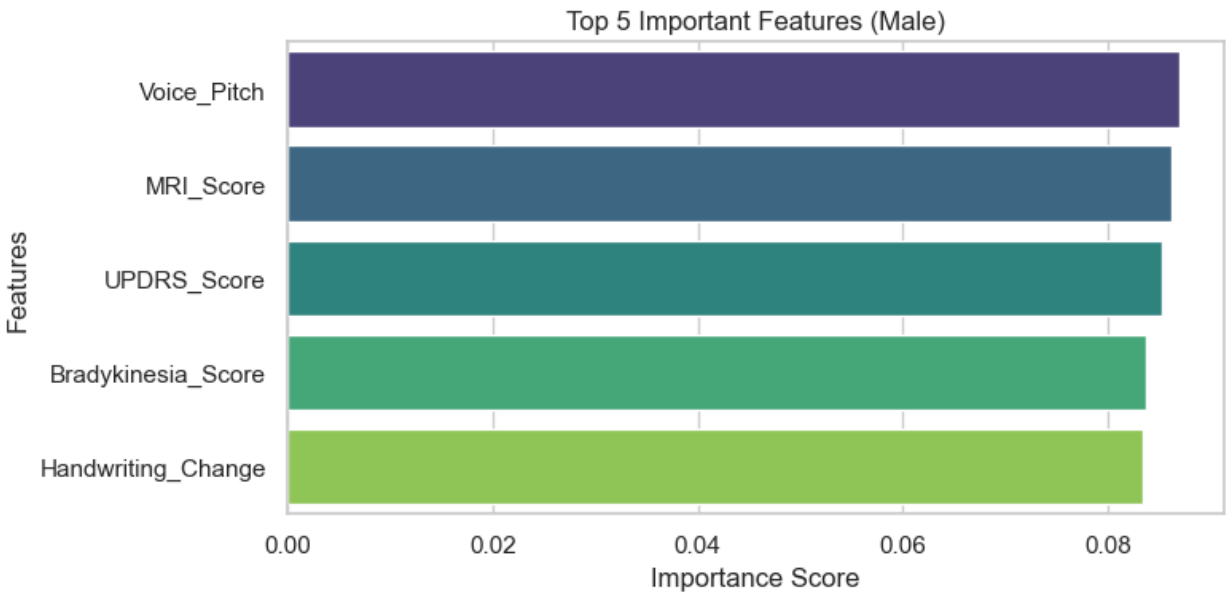
```
# ------------------------------------------------------------
# Gender-wise Modeling of Parkinson's Effect (Top 5 Features Plot)
# ------------------------------------------------------------

# Separate features and target
X = df_processed.drop('Parkinsons_Status', axis=1)
y = df_processed['Parkinsons_Status']

# Split data into Male and Female subsets
X_male = X[X['Sex_Male'] == 1].drop('Sex_Male', axis=1)
y_male = y[X['Sex_Male'] == 1]
```

```
X_female = X[X['Sex_Male'] == 0].drop('Sex_Male', axis=1)
y_female = y[X['Sex_Male'] == 0]

def train_and_evaluate(X, y, gender_label):
    print(f"\n=== {gender_label} Dataset ===")
    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Feature scaling
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Train Random Forest
    rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
    rf_model.fit(X_train_scaled, y_train)

    # Evaluate accuracy
    y_pred = rf_model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    print(f"{gender_label} Test Accuracy: {acc*100:.2f}%")

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f'Confusion Matrix - {gender_label}')
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

    # Feature importance
    feature_importances = pd.Series(rf_model.feature_importances_,
                                    index=X.columns)
    sorted_importances =
feature_importances.sort_values(ascending=False)
    top_features = sorted_importances.head(5)

    # Plot only Top 5
    plt.figure(figsize=(8, 4))
    sns.barplot(
    x=top_features.values,
    y=top_features.index,
    hue=top_features.index,
    palette="viridis",
    legend=False
                )
```

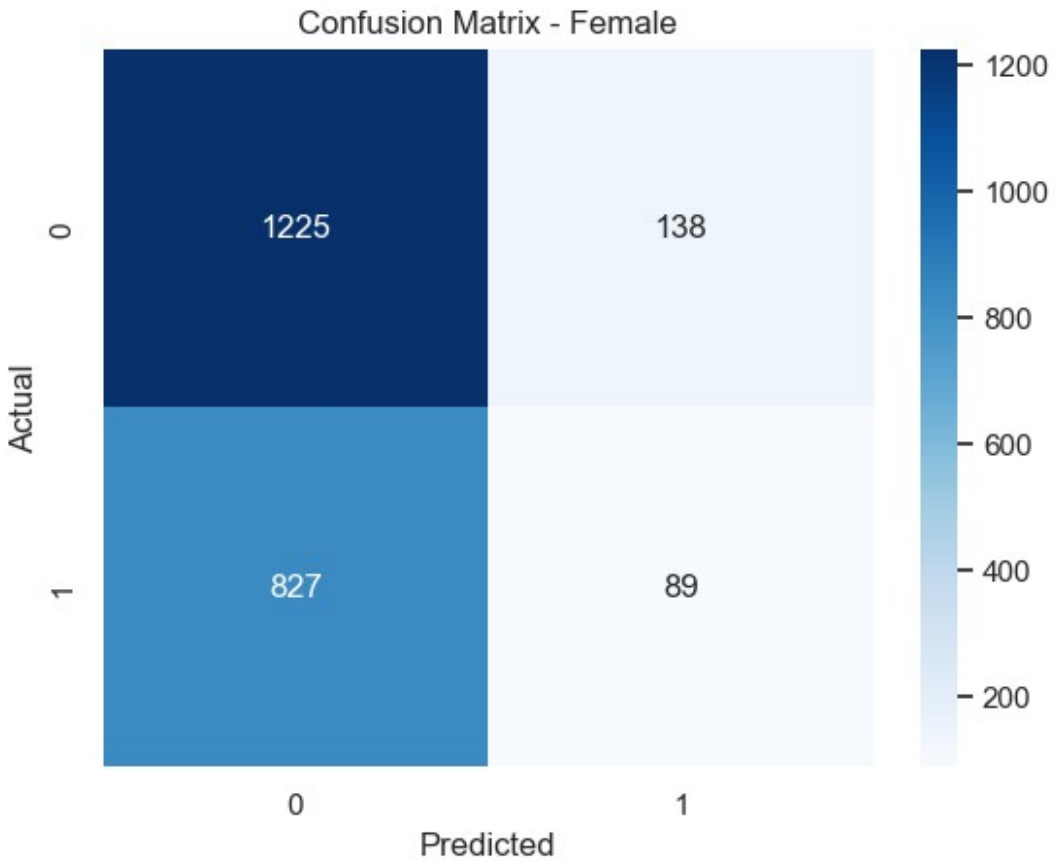
```
    plt.title(f'Top 5 Important Features ({gender_label})')
    plt.xlabel('Importance Score')
    plt.ylabel('Features')
    plt.tight_layout()
    plt.show()


    print(f"\nTop 5 Important Features for {gender_label}:")
    print(top_features)
    return acc

# Run for Male and Female datasets
male_acc = train_and_evaluate(X_male, y_male, "Male")
female_acc = train_and_evaluate(X_female, y_female, "Female")
```

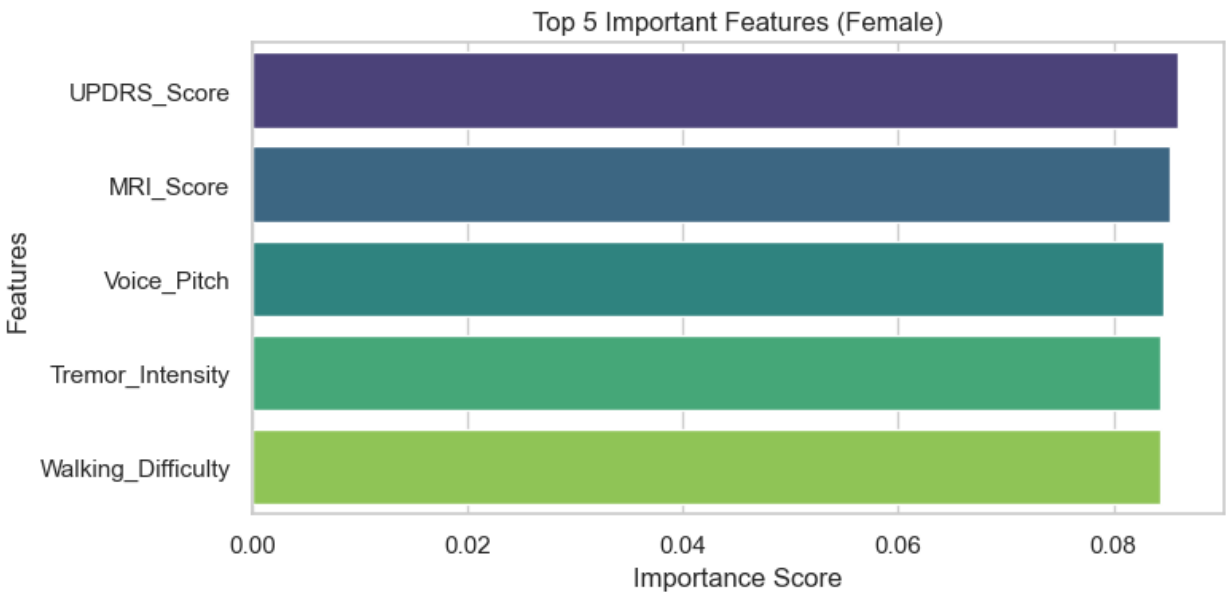
```
=== Male Dataset ===
Male Test Accuracy: 58.10%
```






```
Top 5 Important Features for Male:
Voice_Pitch           0.087036
MRI_Score             0.086328
UPDRS_Score           0.085249
Bradykinesia_Score    0.083745
Handwriting_Change    0.083462
dtype: float64

=== Female Dataset ===
Female Test Accuracy: 57.66%
```






```
Top 5 Important Features for Female:
UPDRS_Score           0.085959
MRI_Score             0.085288
```

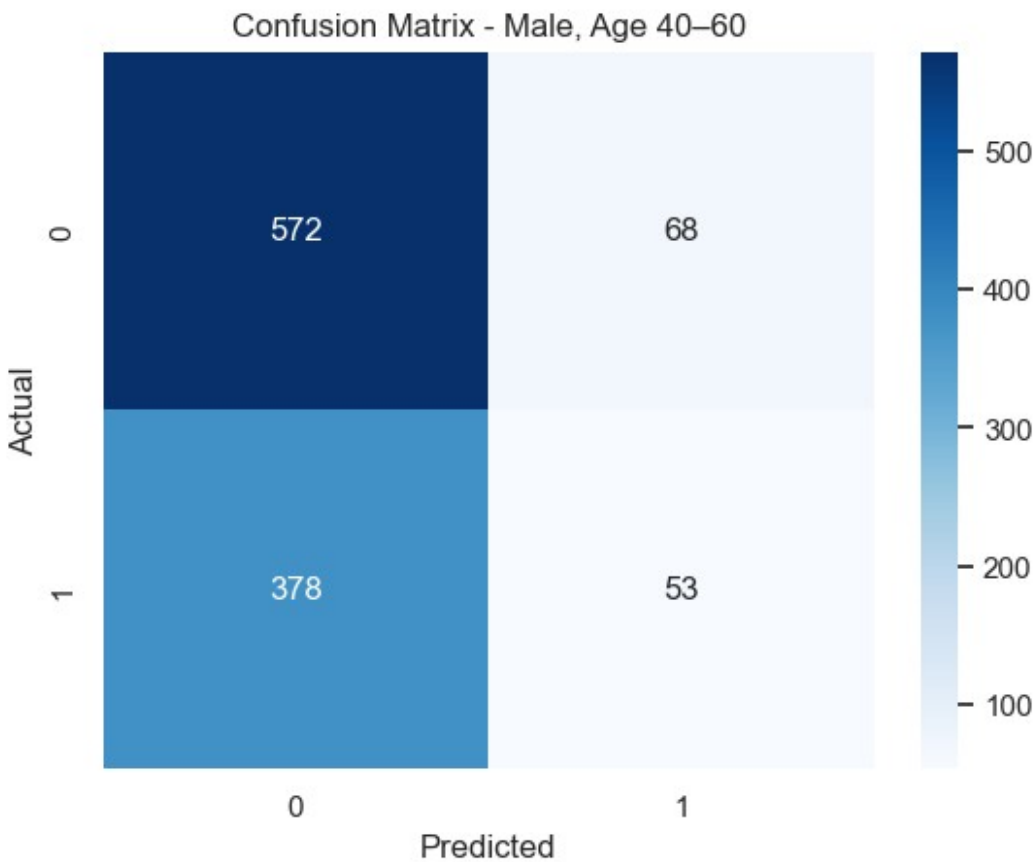
```
Voice_Pitch           0.084706
Tremor_Intensity      0.084390
Walking_Difficulty    0.084335
dtype: float64
```

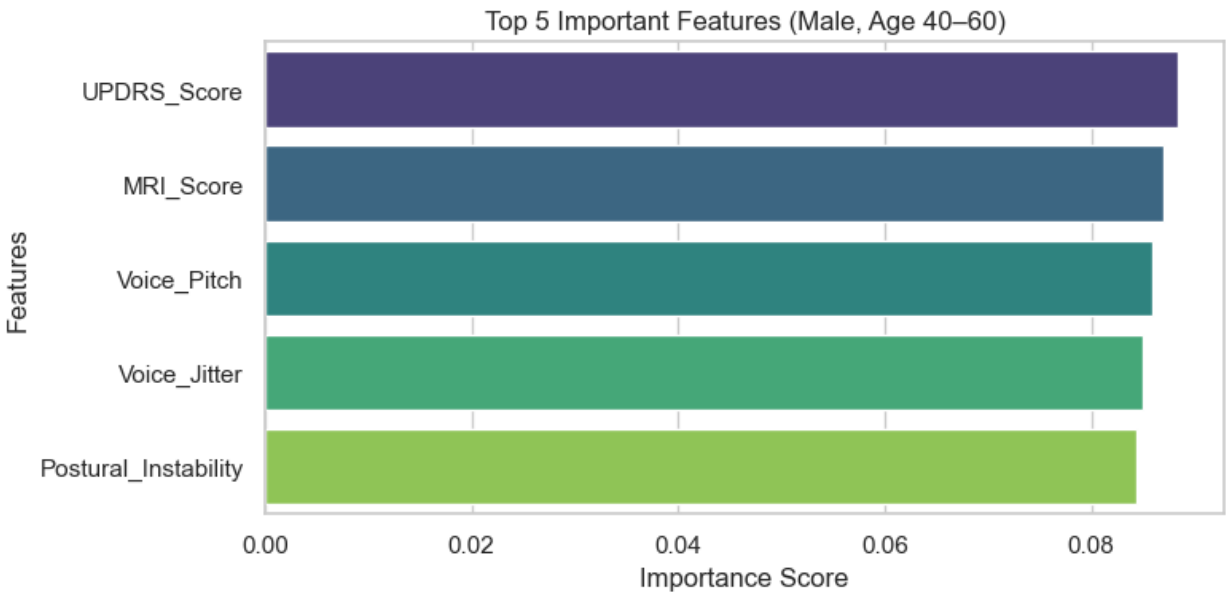
```python
# ------------------------------------------------------------
# Gender + Age Group Analysis (Top 5 Features Only)
# ------------------------------------------------------------

# Create Age Group column
df_processed['Age_Group'] = pd.cut(
    df['Age'],
    bins=[39, 60, 100],  # range 40–60, and 60+
    labels=['40-60', '60+']
)

# Separate features and target
X = df_processed.drop(['Parkinsons_Status'], axis=1)
y = df_processed['Parkinsons_Status']

def train_and_evaluate(X, y, label):
    print(f"\n=== {label} Dataset ===")

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Feature scaling
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Train Random Forest
    rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
    rf_model.fit(X_train_scaled, y_train)

    # Evaluate accuracy
    y_pred = rf_model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    print(f"{label} Test Accuracy: {acc*100:.2f}%")

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f'Confusion Matrix - {label}')
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
```

```
    # Feature importance
    feature_importances = pd.Series(rf_model.feature_importances_,
index=X.columns)
    sorted_importances =
feature_importances.sort_values(ascending=False)
    top_features = sorted_importances.head(5)  # Select only top 5
features

    # Plot only top 5 features (warning-free)
    plt.figure(figsize=(8, 4))
    sns.barplot(
        x=top_features.values,
        y=top_features.index,
        hue=top_features.index,
        palette="viridis",
        legend=False
    )
    plt.title(f'Top 5 Important Features ({label})')
    plt.xlabel('Importance Score')
    plt.ylabel('Features')
    plt.tight_layout()
    plt.show()

    print(f"\nTop 5 Important Features for {label}:")
    print(top_features)
    return acc


# ------------------------------------------------------------
# Run analysis for Gender + Age Group
# ------------------------------------------------------------

# Male, 40–60
X_male_40_60 = X[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '40-60')].drop(['Sex_Male','Age_Group'],
axis=1)
y_male_40_60 = y[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '40-60')]
train_and_evaluate(X_male_40_60, y_male_40_60, "Male, Age 40–60")

# Male, 60+
X_male_60p = X[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '60+')].drop(['Sex_Male','Age_Group'],
axis=1)
y_male_60p = y[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '60+')]
train_and_evaluate(X_male_60p, y_male_60p, "Male, Age 60+")

# Female, 40–60
```

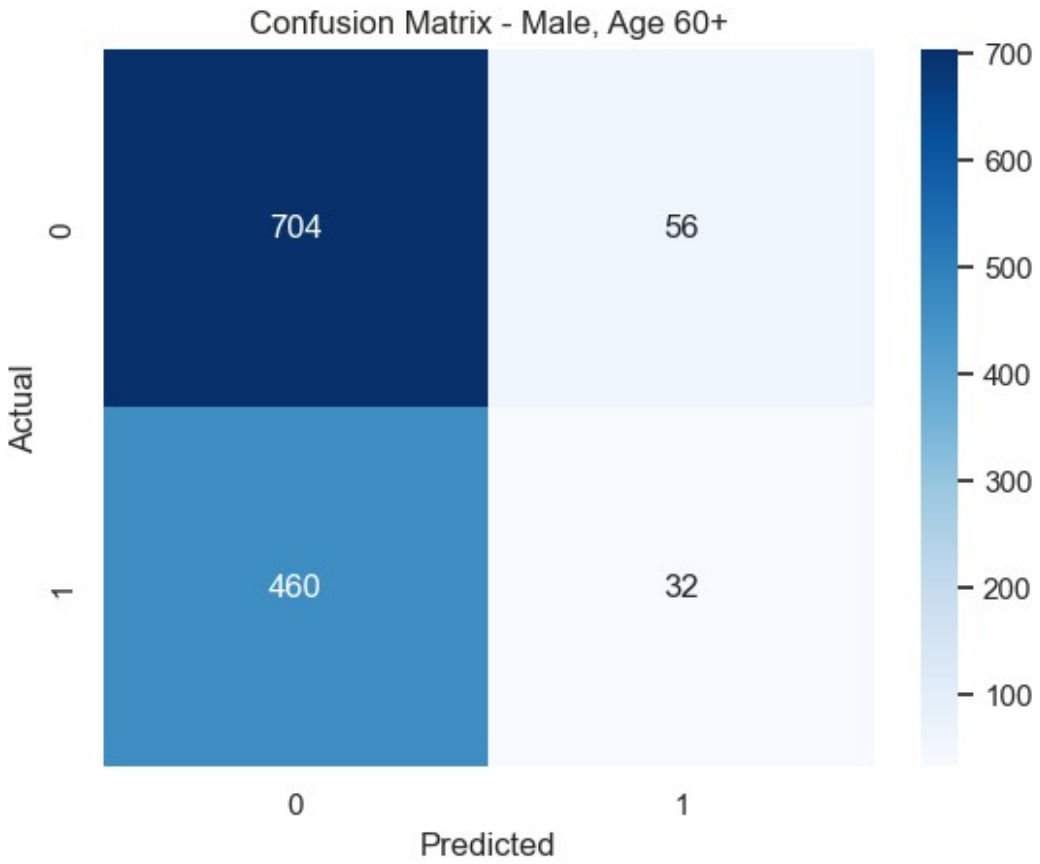
```
X_female_40_60 = X[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '40-60')].drop(['Sex_Male','Age_Group'],
axis=1)
y_female_40_60 = y[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '40-60')]
train_and_evaluate(X_female_40_60, y_female_40_60, "Female, Age 40–
60")

# Female, 60+
X_female_60p = X[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '60+')].drop(['Sex_Male','Age_Group'],
axis=1)
y_female_60p = y[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '60+')]
train_and_evaluate(X_female_60p, y_female_60p, "Female, Age 60+")
```

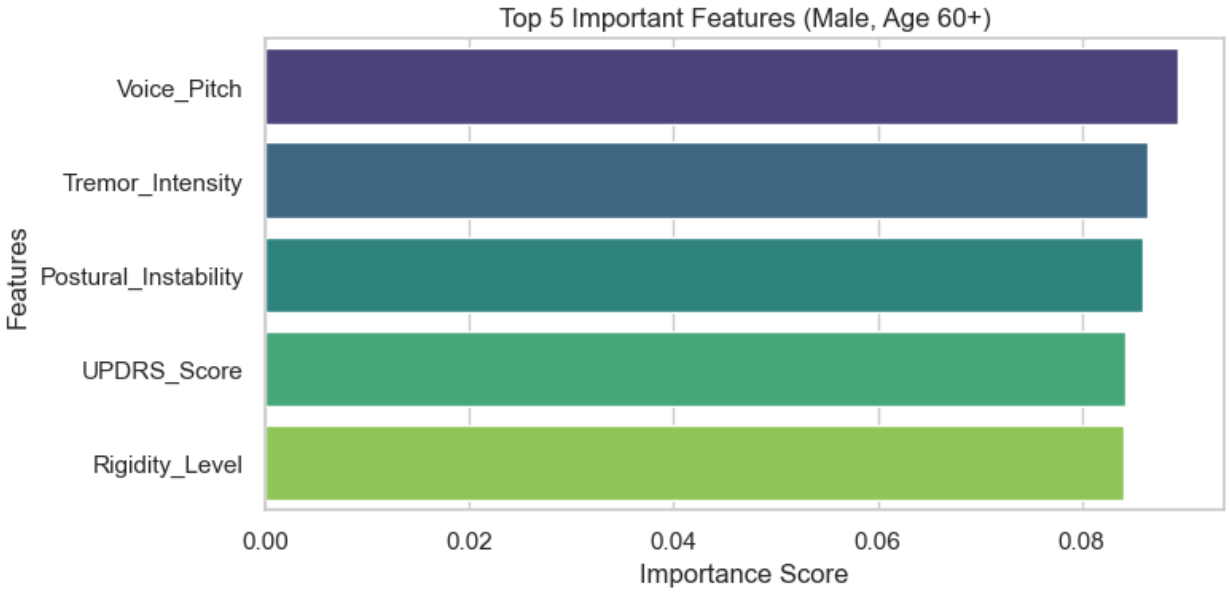
```
=== Male, Age 40–60 Dataset ===
Male, Age 40–60 Test Accuracy: 58.36%
```





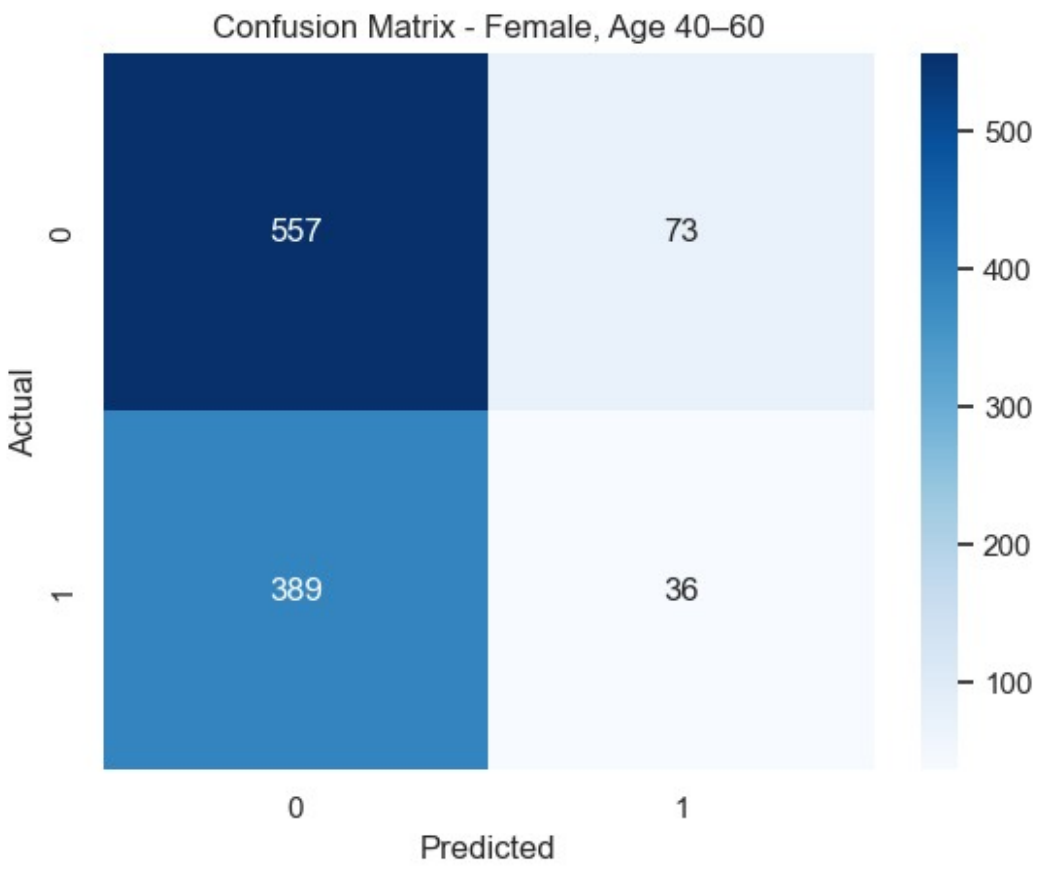

```
Top 5 Important Features for Male, Age 40–60:
UPDRS_Score             0.088436
MRI_Score               0.087015
Voice_Pitch             0.085902
Voice_Jitter            0.085032
Postural_Instability    0.084417
dtype: float64

=== Male, Age 60+ Dataset ===
Male, Age 60+ Test Accuracy: 58.79%
```

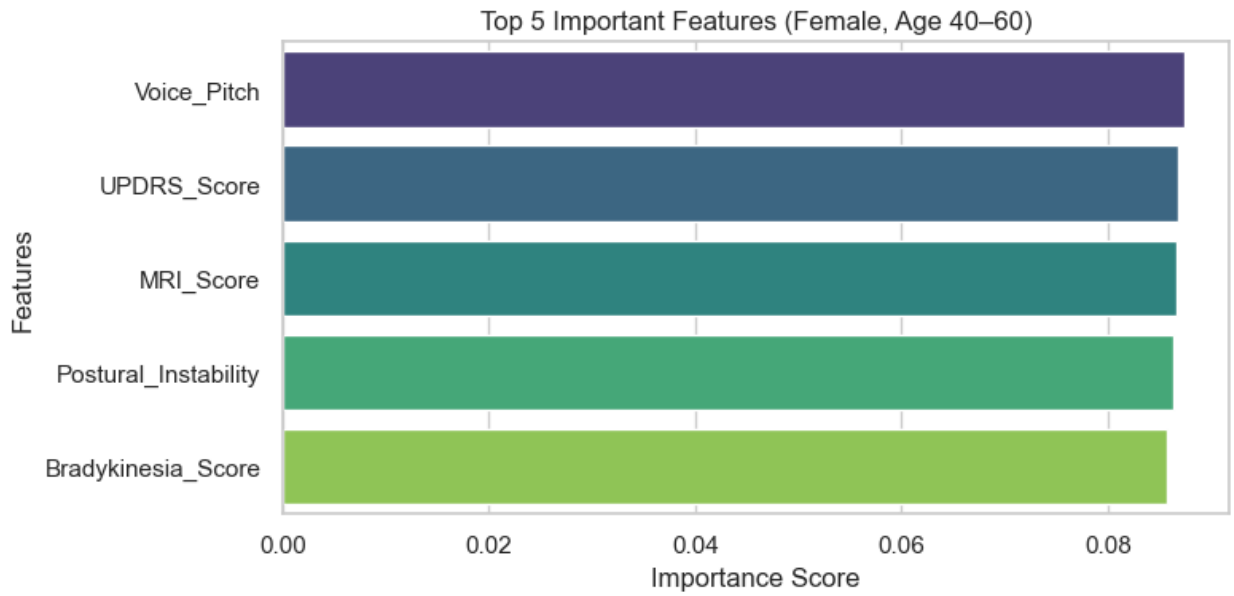




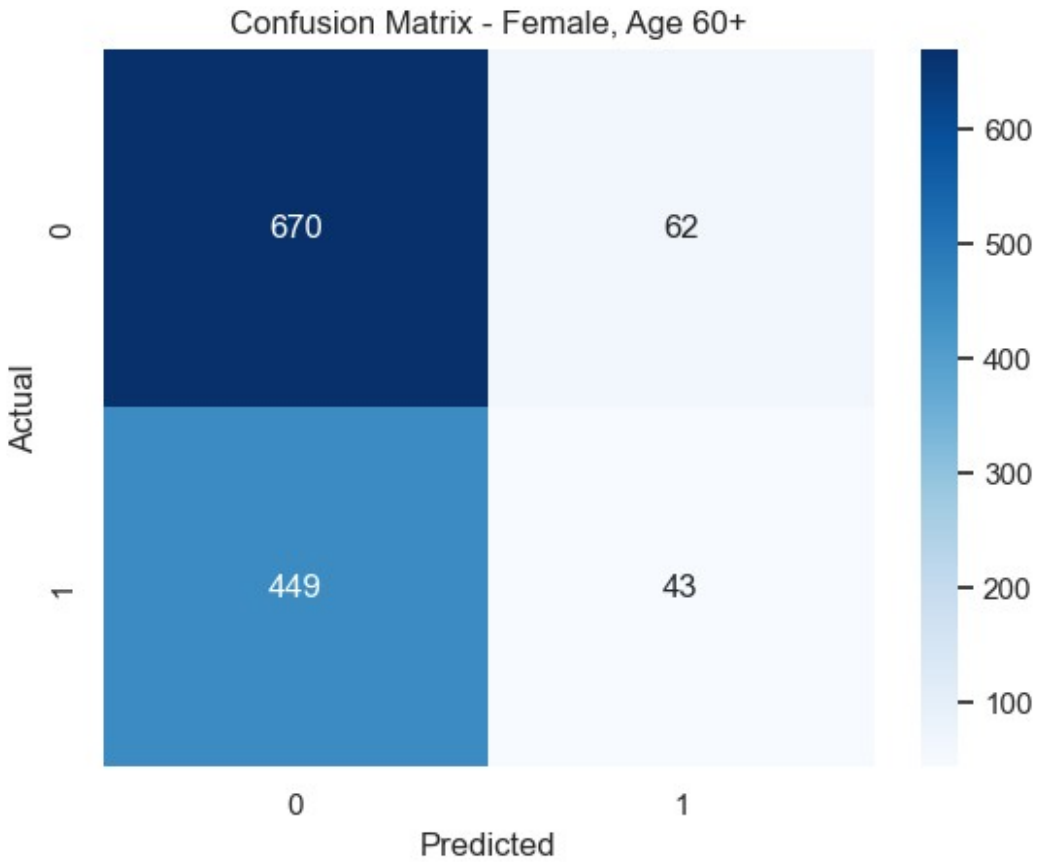

```
Top 5 Important Features for Male, Age 60+:
Voice_Pitch             0.089466
Tremor_Intensity        0.086424
```

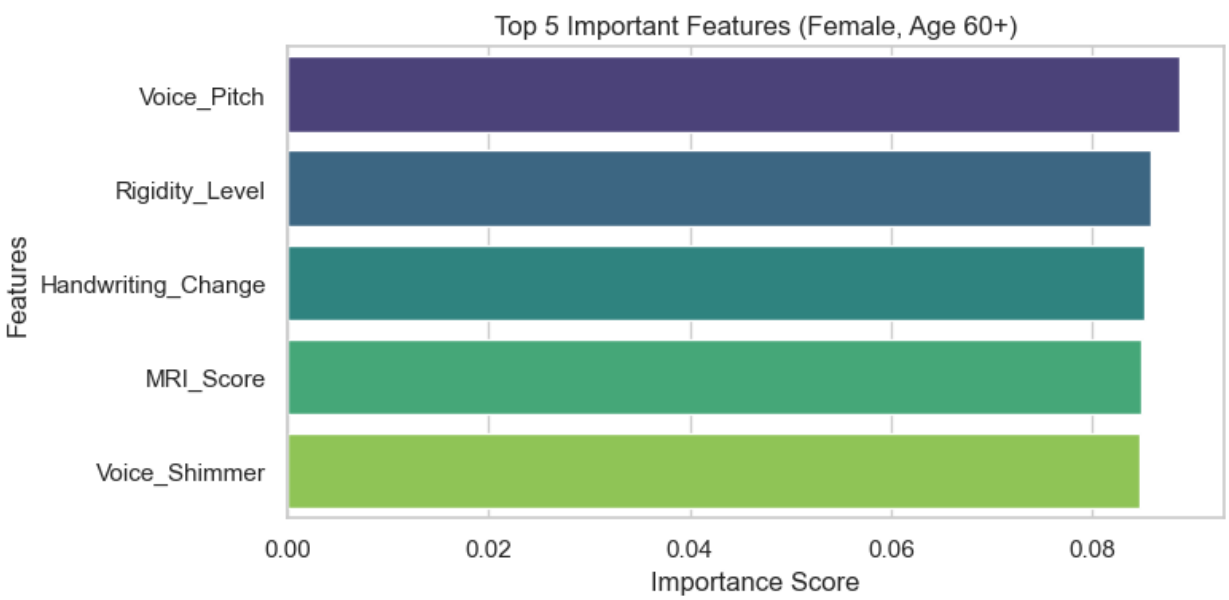
```
Postural_Instability    0.086004
UPDRS_Score             0.084203
Rigidity_Level          0.084038
dtype: float64

=== Female, Age 40–60 Dataset ===
Female, Age 40–60 Test Accuracy: 56.21%
```






```
Top 5 Important Features for Female, Age 40–60:
Voice_Pitch             0.087376
UPDRS_Score             0.086784
MRI_Score               0.086598
Postural_Instability    0.086325
Bradykinesia_Score      0.085704
dtype: float64

=== Female, Age 60+ Dataset ===
Female, Age 60+ Test Accuracy: 58.25%
```






```
Top 5 Important Features for Female, Age 60+:
Voice_Pitch           0.088721
Rigidity_Level        0.085824
```

```
Handwriting_Change    0.085248
MRI_Score             0.084977
Voice_Shimmer         0.084833
dtype: float64
```

```
0.5825163398692811
```

```
# ------------------------------------------------------------
# Gender + Age Group Analysis with Summary Table
# ------------------------------------------------------------

results = []  # to store summary

def train_and_evaluate_summary(X, y, label):
    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Feature scaling
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Train Random Forest
    rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
    rf_model.fit(X_train_scaled, y_train)

    # Evaluate
    y_pred = rf_model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred) * 100

    # Feature importance
    feature_importances = pd.Series(rf_model.feature_importances_,
                                    index=X.columns)
    sorted_importances =
feature_importances.sort_values(ascending=False)
    top_features = sorted_importances.head(3).index.tolist()

    # Append to results
    results.append([label, round(acc, 2), top_features])

# ------------------------------------------------------------
# Run analysis for each group
# ------------------------------------------------------------

# Male, 40–60
X_male_40_60 = X[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '40-60')].drop(['Sex_Male','Age_Group'],
axis=1)
```

```
y_male_40_60 = y[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '40-60')]
train_and_evaluate_summary(X_male_40_60, y_male_40_60, "Male, Age 40-
60")

# Male, 60+
X_male_60p = X[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '60+')].drop(['Sex_Male','Age_Group'],
axis=1)
y_male_60p = y[(df_processed['Sex_Male'] == 1) &
(df_processed['Age_Group'] == '60+')]
train_and_evaluate_summary(X_male_60p, y_male_60p, "Male, Age 60+")

# Female, 40–60
X_female_40_60 = X[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '40-60')].drop(['Sex_Male','Age_Group'],
axis=1)
y_female_40_60 = y[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '40-60')]
train_and_evaluate_summary(X_female_40_60, y_female_40_60, "Female,
Age 40-60")

# Female, 60+
X_female_60p = X[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '60+')].drop(['Sex_Male','Age_Group'],
axis=1)
y_female_60p = y[(df_processed['Sex_Male'] == 0) &
(df_processed['Age_Group'] == '60+')]
train_and_evaluate_summary(X_female_60p, y_female_60p, "Female, Age
60+")

# ------------------------------------------------------------
# Create a summary DataFrame
# ------------------------------------------------------------
summary_df = pd.DataFrame(results, columns=["Group", "Accuracy (%)",
"Top 3 Features"])
print("\n==== Summary Results by Gender & Age Group ====\n")
print(summary_df.to_string(index=False))
```

```
==== Summary Results by Gender & Age Group ====

            Group  Accuracy (%)
Top 3 Features
  Male, Age 40-60         58.36                 [UPDRS_Score,
MRI_Score, Voice_Pitch]
    Male, Age 60+         58.79 [Voice_Pitch, Tremor_Intensity,
Postural_Instability]
Female, Age 40-60         56.21                 [Voice_Pitch,
UPDRS_Score, MRI_Score]
```

```
  Female, Age 60+          58.25     [Voice_Pitch, Rigidity_Level,
Handwriting_Change]
```